Function Call by Value in C

In C language, a function can be called from any other function, including itself. There are two ways in which a function can be called − (a) Call by Value and (b) Call by Reference. By default, the Call by Value mechanism is employed.

Formal Arguments and Actual Arguments

You must know the terminologies to understand how the call by value method works −

Formal arguments − A function needs certain data to perform its desired process. When a function is defined, it is assumed that the data values will be provided in the form of parameter or argument list inside the parenthesis in front of the function name. These arguments are the variables of a certain data type.

Actual arguments − When a certain function is to be called, it should be provided with the required number of values of the same type and in the same sequence as used in its definition.

Take a look at the following snippet −

```
type function_name(type var1, type var2, ...)
```

Here, the argument variables are called the **formal arguments**. Inside the function's scope, these variables act as its **local variables**.

Consider the following function −

```
int add(int x, int y){

   int z = x + y;

   return z;

}
```

The arguments **x** and **y** in this function definition are the formal arguments.

Example: Call by Value in C

If the **add()** function is called, as shown in the code below, then the variables inside the parenthesis (**a** and **b**) are the actual arguments. They are passed to the function.

Take a look at the following example –

</> Open Compiler

```
#include <stdio.h>

int add(int x, int y){

   int z = x + y;

   return z;
}

int main(){

   int a = 10, b = 20;

   int c = add(a, b);

   printf("Addition: %d", c);
}
```

Output

When you run this code, it will produce the following output –

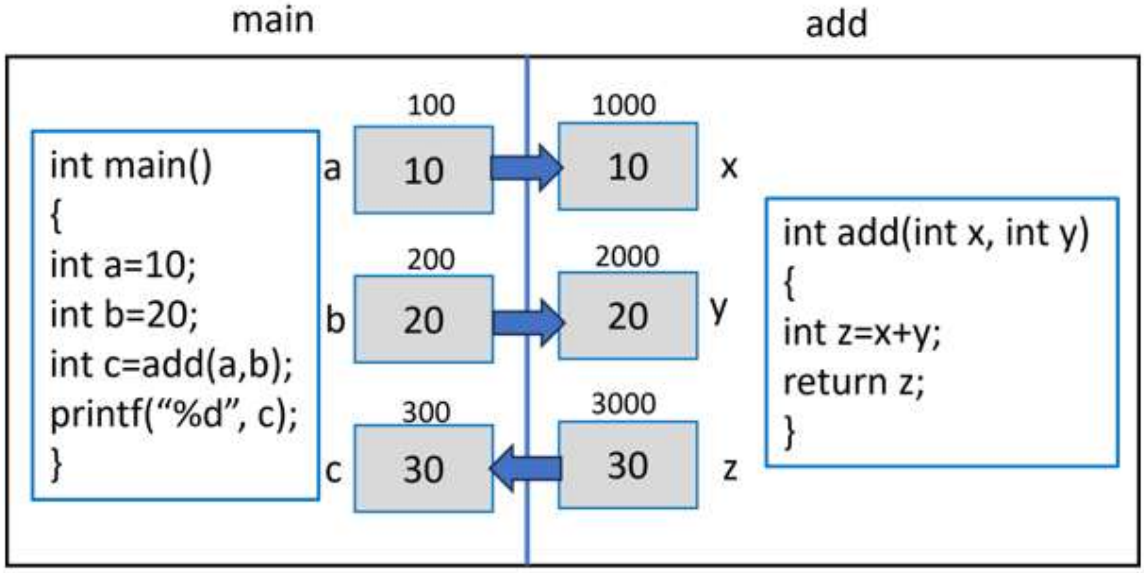
```
Addition: 30
```

The Call by Value method implies that the values of the actual arguments are copied in the formal argument variables. Hence, "x" takes the value of "a" and "b" is assigned to "y". The local variable "z" inside the add() function stores the addition value. In the main() function, the value returned by the add() function is assigned to "c", which is printed.

Note that a variable in C language is a named location in the memory. Hence, variables are created in the memory and each variable is assigned a random memory address by the compiler.

How Does "Call by Value" Work in C?

Let us assume that the variables **a**, **b** and **c** in the main() function occupy the memory locations 100, 200 and 300 respectively. When the add() function is called with **a** and **b** as actual arguments, their values are stored in **x** and **y** respectively.



The variables **x**, **y**, and **z** are the local variables of the add() function. In the memory, they will be assigned some random location. Let's assume that they are created in memory address 1000, 2000 and 3000, respectively.

Since the function is called by copying the value of the actual arguments to their corresponding formal argument variables, the locations 1000 and 2000 which are the address of **x** and **y** will hold 10 and 20, respectively. The compiler assigns their addition to the third local variable **z** which is returned.

As the control comes back to the main() function, the returned data is assigned to **c**, which is displayed as the output of the program.

Example

Call by Value is the default function calling mechanism in C. It eliminates a function's potential side effects, making your software simpler to maintain and easy to understand. It is best suited for a function expected to do a certain computation on the argument received and return the result.

Here is another example of Call by Value –

</> Open Compiler

```
#include <stdio.h>

/* function declaration */
```

```
void swap(int x, int y);

int main(){

   /* local variable definition */
   int a = 100;
   int b = 200;

   printf("Before swap, value of a: %d\n", a);
   printf("Before swap, value of b: %d\n", b);

   /* calling a function to swap the values */
   swap(a, b);

   printf("After swap, value of a: %d\n", a);
   printf("After swap, value of b: %d\n", b);

   return 0;
}

void swap(int x, int y){

   int temp;

   temp = x; /* save the value of x */
   x = y;    /* put y into x */
   y = temp; /* put temp into y */

   return;
}
```

Output

When you run this code, it will produce the following output –

```
Before swap, value of a: 100
Before swap, value of b: 200
After swap, value of a: 100
After swap, value of b: 200
```

It shows that there are no changes in the values, although they had been changed inside the function.

Since the values are copied in different local variables of another function, any manipulation doesn't have any effect on the actual argument variables in the calling function.

However, the Call by Value method is less efficient when we need to pass large objects such as an array or a file to another function. Also, in some cases, we may need the actual arguments to be manipulated by another function. In such cases, the Call by Value mechanism is not useful. We have to explore the Call by Reference mechanism for that purpose. Refer the next chapter for a detailed explanation on the Call by Reference mechanism of C.

The Call by Reference approach involves passing the address of the variable holding the value of the actual argument. You can devise a calling method that is a mix of Call by Value and Call by Reference. In this case, some arguments are passed by value and others by reference.

Function Call by Reference in C

There are two ways in which a function can be called: (a) Call by Value and (b) Call by Reference. In this chapter, we will explain the mechanism of calling a function by reference.

Let us start this chapter with a brief overview on "pointers" and the "address operator (&)". It is important that you learn these two concepts in order to fully understand the mechanism of Call by Reference.

The Address Operator (&) in C

In C language, a variable is a named memory location. When a variable declared, the compiler allocates a random location in the memory and internally identifies the location with the user-defined name.

To fetch the address at which the variable has been created, we use the **address (&) operator**.

Example

Take a look at the following example –

</> Open Compiler

```
#include <stdio.h>

int main(){

   int x = 10;

   printf("x: %d Address of x: %d", x, &x);
}
```

Output

This will print the value of **x** and its address –

```
x: 10 Address of x: -1990957196
```

What is a Pointer in C?

A pointer is a variable that stores the address of another variable. To declare a pointer variable, its name is prefixed with the * symbol. The type of the pointer variable and its host variable must be same.

The address is assigned with the **&** operator. The **dereference operator (*)** is used with the pointer. It fetches the value of a variable whose address is assigned to the pointer.

Example

The following example demonstrates how referening and dereferencing work in C –

</> Open Compiler

```
#include <stdio.h>

int main(){

   int x = 10;
   int *y = &x;

   printf("x: %d Address of x: %d\n", x, &x);
   printf("Address of y: %d \n", &y);
   printf("Value at address in y: %d\n", *y);
}
```

Output

Run the code and check its output –

```
x: 10 Address of x: -1742755108
Address of y: -1742755104
Value at address in y: 10
```


How Does Call by Reference Work in C?

When a function is called by reference, the address of the actual argument variables passed, instead of their values.

Let us define the add() function that receives the references of two variables –

```
int add(int *x, int *y){

   int z = *x + *y;

   return z;
}
```

When such a function is called, we pass the address of the actual argument.

Example

Let us call the add() function by reference from inside the main() function –

</> Open Compiler

```
#include <stdio.h>

/* function declaration */
int add(int *, int *);

int main(){

   int a = 10, b = 20;
   int c = add(&a, &b);

   printf("Addition: %d", c);
}

int add(int *x, int *y){

   int z = *x + *y;

   return z;
}
```

Output

When you run this code, it will produce the following output –

```
Addition: 30
```

Now let's understand how this code actually works. The main() function passes the address of **a** and **b** to the **add()** function. The addresses of **a** and **b** are assigned to the pointer variables **x** and **y**.

Now focus on the statement "z = *x + *y;" inside the add() function. Remember that **x** stores the address of **a**. The dereference operator in ***x** and ***y** fetches the values of **a** and **b** respectively, hence **z** is the addition of **a** and **b** in the main() function.

Example: Swap Values with Call by Reference

Let us understand in more detail how the Call by Reference mechanism works, with the help of the following example that interchanges value of two variables.

</> Open Compiler

```
#include <stdio.h>

/* Function definition to swap the values */
/* It receives the reference of two variables whose values are to be swapped
*/

int swap(int *x, int *y){

   int z;

   z = *x;   /* save the value at address x */
   *x = *y;  /* put y into x */
   *y = z;   /* put z into y */

   return 0;
}

/* The main() function has two variables "a" and "b" */
/* Their addresses are passed as arguments to the swap() function. */

int main(){
```

```
   /* local variable definition */
   int a = 10;
   int b = 20;

   printf("Before swap, value of a: %d\n", a );
   printf("Before swap, value of b: %d\n", b );

   /* calling a function to swap the values */
   swap(&a, &b);

   printf("After swap, value of a: %d\n", a);
   printf("After swap, value of b: %d\n", b);

   return 0;
}
```

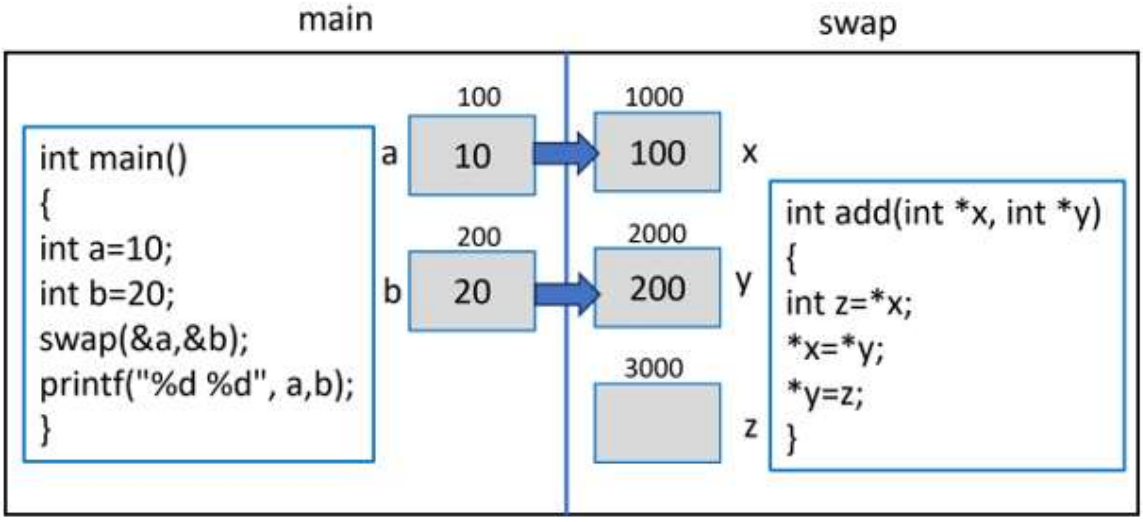
Output

When you run this code, it will produce the following output –

```
Before swap, value of a: 10
Before swap, value of b: 20
After swap, value of a: 20
After swap, value of b: 10
```

Explanation

Assume that the variables **a** and **b** in the main() function are allotted locations with the memory address 100 and 200 respectively. As their addresses are passed to **x** and **y** (remember that they are pointers), the variables **x**, **y** and **z** in the swap() function are created at addresses 1000, 2000 and 3000 respectively.

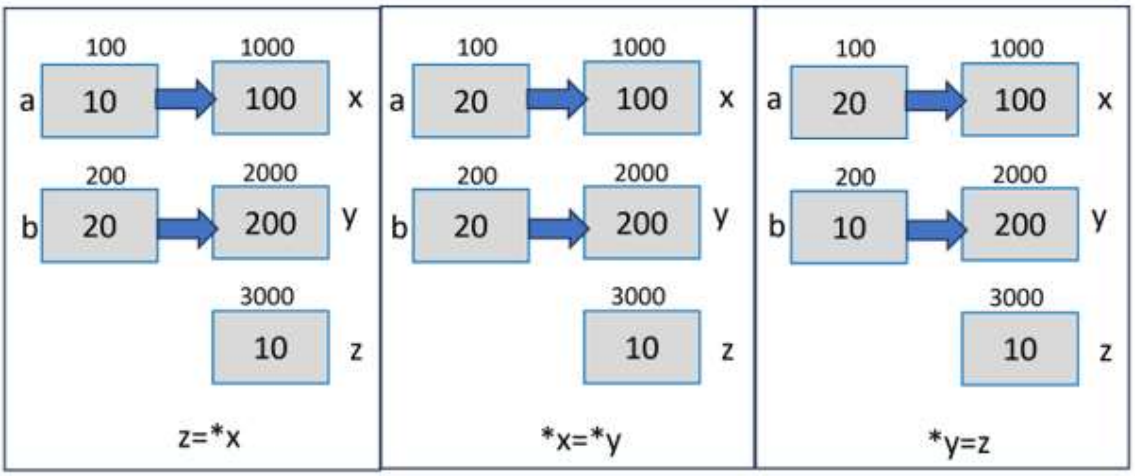


Since "x" and "y" store the address of "a" and "b", "x" becomes 100 and "y" becomes 200, as the above figure shows.

Inside the swap() function, the first statement "**z = *x**" causes the value at address in "x" to be stored in "x" (which is 10). Similarly, in the statement "***x = *y;**", the value at the address in "y" (which is 20) is stored in the location whose pointer is "x".

Finally, the statement "***y = z;**" assigns the "z" to the variable pointed to by "y", which is "b" in the main() function. The values of "a" and "b" now get swapped.

The following figure visually demonstrates how it works –



Mixing Call by Value and Call by Reference

You can use a function calling mechanism that is a combination of Call by Value and Call by Reference. It can be termed as "mixed calling mechanism", where some of the arguments are passed by value and others by reference.

A function in C can have more than one arguments, but can return only one value. The Call by Reference mechanism is a good solution to overcome this restriction.

Example

In this example, the calculate() function receives an integer argument by value, and two pointers where its square and cube are stored.

</> Open Compiler

```
#include <stdio.h>
#include <math.h>

/* function declaration */
int calculate(int, int *, int *);

int main(){

   int a = 10;
   int b, c;

   calculate(a, &b, &c);

   printf("a: %d \nSquare of a: %d \nCube of a: %d", a, b, c);
}

int calculate(int x, int *y, int *z){

   *y  = pow(x,2);
   *z = pow(x, 3);

   return 0;
}
```

Output

When you run this code, it will produce the following output –

```
a: 10
Square of a: 100
Cube of a: 1000
```

The Call by Reference mechanism is widely used when a function needs to perform memory-level manipulations such as controlling the peripheral devices, performing dynamic allocation, etc.